

MDX File Format Specification

Format: Blizzard MDX Model Format (Warcraft III / Warcraft III: Reforged) **Byte Order:** Little-endian **File Magic:** **MDLX** **Supported Versions:** 800 (Classic), 900–1200 (Reforged) **Companion format:** **MDL** — the equivalent human-readable text format

This document describes the MDX format used by Warcraft III (Classic and Reforged). The format is chunk-based and supports skeletal animation, particle effects, cameras, and collision volumes.

Authorship attribution: Fernando A. Sahmkow

Revision History

Date	Version	Description
2025-07-18	2.0	Complete rewrite based on WhiteoutLib parser/writer implementation, GhostWolf's HiveWorkshop reference, and validation against 5081-file Reforged corpus
2026-05-21	2.1	Corrected the Layer fresnel-field version gate (v1000, not v900); documented the v≥1100 layer shader field as a ShaderType enum (was mislabelled isHD); fixed the KEVT field order (globalSequenceId precedes the track times — v2.0 had it backwards); added field defaults, signed keyframe-time notes, and the precise version→release mapping. Validated by a byte-exact MDX→MDL→MDX round-trip over the full war3.w3mod game-asset corpus.

Table of Contents

- 1. Overview
- 2. Conventions
- 3. File Structure
- 4. Chunk Header
- 5. Parsing Strategy
- 6. Common Structures
- 7. Top-Level Chunks
- 8. Animation Tracks
- 9. Node Structure
- 10. Version Differences
- 11. Implementation Notes

Appendices

- A — PopcornFX Integration
- Credits & References

1. Overview

MDX is a **binary 3D model format** created by Blizzard Entertainment for Warcraft III and Warcraft III: Reforged. It stores geometry, materials, skeletal animation, particle effects, cameras, collision volumes, and other data needed to render and animate game models.

The format is **chunk-based**: the file consists of a 4-byte magic number followed by a sequence of tagged chunks in no prescribed order. All chunks are optional. All multi-byte values are **little-endian**.

Known Versions

Version	Game
800	Warcraft III: Reign of Chaos / The Frozen Throne (Classic)
900	Warcraft III: Reforged — Beta
1000	Warcraft III: Reforged — First Release (1.32)
1100	Warcraft III: Reforged — 2.0.0 and later
1200	Warcraft III: Reforged — current 2.0.x

Corpus note: A corpus of 5081 MDX files extracted from Warcraft III: Reforged consists entirely of version 1200 files. Classic v800 models exist in older game data.

2. Conventions

Data Types

Notation	Size	Description
u8	1	Unsigned 8-bit integer
u16	2	Unsigned 16-bit integer
u32	4	Unsigned 32-bit integer
i32	4	Signed 32-bit integer
f32	4	IEEE 754 single-precision float
char[N]	N	Fixed-length null-padded ASCII string
float[N]	4×N	Array of N f32 values
FourCC	4	Four ASCII characters read as a u32

Notation

- **(X)** means field or sub-chunk X is **optional** and may or may not be present.
- **[N]** after a type means an array of N elements.
- **[?]** means the array count is not stored directly; see parsing notes.
- **// version > V** means the field only exists when the model version exceeds V.
- Flag fields use hexadecimal bit masks (e.g., **0x1**, **0x2**).

FourCC Tags

Chunk identifiers are 4-byte ASCII strings stored as little-endian `u32` values. For example, "MDLX" is the bytes `0x4D, 0x44, 0x4C, 0x58`, which as a little-endian `u32` is `0x584C444D`. In code, define constants like:

```
constexpr u32 makeTag(char a, char b, char c, char d) {
    return static_cast<u32>(a) | (static_cast<u32>(b) << 8) |
           (static_cast<u32>(c) << 16) | (static_cast<u32>(d) << 24);
}
constexpr u32 MDLX_TAG = makeTag('M', 'D', 'L', 'X');
```

3. File Structure

```
MDX {
    FourCC  "MDLX"           // Magic number (0x584C444D)
    (VERS)                               // Version
    (MODL)                               // Model info
    (SEQS)                               // Animation sequences
    (GLBS)                               // Global sequences
    (TEXTS)                             // Textures
    (SNDS)                               // Sounds (deprecated, unused in shipped models)
    (SNEM)                               // Sound emitters (deprecated, unused in shipped
models)
    (MTLS)                               // Materials
    (TXAN)                               // Texture animations
    (GEOS)                               // Geosets (mesh data)
    (GEOA)                               // Geoset animations
    (BONE)                               // Bones
    (LITE)                               // Lights
    (HELP)                               // Helpers
    (ATCH)                               // Attachments
    (PIVT)                               // Pivot points
    (PREM)                               // Particle emitters (model-based)
    (PRE2)                               // Particle emitters 2 (quad-based)
    (RIBB)                               // Ribbon emitters
    (EVTS)                               // Event objects
    (CAMS)                               // Cameras
    (CLID)                               // Collision shapes
    (BPOS)                               // Bind poses (version > 800)
    (FAFX)                               // Face effects (version > 800)
    (CORN)                               // Popcorn emitters (version > 800)
}
```

All chunks are optional and may appear in any order. A valid MDX file can consist of just the 4-byte "MDLX" magic. Unknown chunks should be skipped using the size from the chunk header.

4. Chunk Header

Every top-level chunk (and many sub-chunks) begins with an 8-byte header:

```
ChunkHeader {
    FourCC  tag           // 4-byte ASCII identifier
    u32     size          // Size in bytes of chunk data (excludes this 8-byte
header)
}
```

To skip an unknown chunk, read the header and advance **size** bytes.

5. Parsing Strategy

```
read magic "MDLX" (4 bytes)

while (bytesRemaining > 0) {
    tag = readU32()    // FourCC
    size = readU32()   // Chunk data size
    endPos = currentPos + size

    switch (tag) {
        case VERS_TAG: parseVERS(size); break;
        case MODL_TAG: parseMODL(size); break;
        // ... handle known chunks ...
        default: skip(size); break;    // Unknown chunk
    }

    assert(currentPos == endPos)    // Validate exact consumption
}
```

Variable-Sized Object Chunks

Many chunks contain arrays of variable-sized objects. Two patterns exist:

Pattern 1 — Objects with **inclusiveSize:** Materials, layers, geosets, geoset animations, lights, attachments, particle emitters, particle emitter 2s, ribbon emitters, cameras, texture animations, and corn emitters store an **inclusiveSize** at the start of each object. This size includes the 4 bytes of the **inclusiveSize** field itself.

```
totalRead = 0
while (totalRead < chunkSize) {
    inclusiveSize = readU32()
    parseObject(inclusiveSize - 4)    // Remaining bytes
    totalRead += inclusiveSize
}
```

Pattern 2 — Objects without `inclusiveSize`: Bones, event objects, helpers, and collision shapes do **not** have an explicit `inclusiveSize`. Their size must be computed:

- **Bone:** Node `inclusiveSize` + 8 bytes (geosetId + geosetAnimationId)
- **Helper:** Node `inclusiveSize` exactly
- **Event Object:** Node `inclusiveSize` + KEVT sub-chunk size (if present)
- **Collision Shape:** Node `inclusiveSize` + type-dependent vertex data + optional radius

6. Common Structures

6.1 Extent

Size: **28 bytes**

```
Extent {
    f32      boundsRadius
    float[3] minimum      // Bounding box min (x, y, z)
    float[3] maximum      // Bounding box max (x, y, z)
}
```

6.2 Node

See [Section 9 — Node Structure](#).

7. Top-Level Chunks

7.1 VERS — Version

Tag: `0x53524556` · **Fixed size:** 4 bytes

```
VERS {
    u32      version      // 800, 900, 1000, 1100, or 1200
}
```

7.2 MODL — Model Info

Tag: `0x4C444F4D` · **Fixed size:** 372 bytes

```
MODL {
    char[80]  name          // Model name
    char[260] animationFileName // Animation file path
    Extent    extent        // Model bounds (28 bytes)
    u32       blendTime     // Animation blend time (ms)
}
```

7.3 SEQS — Sequences

Tag: 0x53514553 · **Element size:** 132 bytes · **Count:** size / 132

```
SEQS {
    Sequence[size / 132] sequences
}

Sequence {
    char[80]    name           // 132 bytes
    u32         intervalStart  // Start time (ms)
    u32         intervalEnd    // End time (ms)
    f32         moveSpeed
    u32         flags          // 0 = looping, 1 = non-looping
    f32         rarity          // Playback probability weight
    u32         syncPoint
    Extent      extent         // 28 bytes
}
```

7.4 GLBS — Global Sequences

Tag: 0x53424C47 · **Element size:** 4 bytes · **Count:** size / 4

```
GLBS {
    u32[size / 4] durations    // Duration of each global sequence (ms)
}
```

7.5 TEXS — Textures

Tag: 0x53584554 · **Element size:** 268 bytes · **Count:** size / 268

```
TEXS {
    Texture[size / 268] textures
}

Texture {
    u32         replaceableId  // 268 bytes
    etc.        // 0 = none, 1 = team color, 2 = team glow,
    char[260]   fileName       // Texture file path (.blp / .tga)
    u32         flags          // 0x1 = wrap width, 0x2 = wrap height
}
```

7.6 SNDS — Sounds

Tag: 0x53444E53 · **Element size:** 56 bytes · **Count:** size / 56

Note: This chunk was used before Warcraft III released and is not present in any known shipped model. Readers for it still exist in the internal game code. Documented for completeness based on disassembly research.

```
SNDS {
    Sound[size / 56] sounds
}

Sound {
    // 56 bytes
    char[44]    soundFile    // Path to sound file (null-terminated)
    f32         maximumDistance // Maximum audible distance
    f32         minimumDistance // Minimum distance (full volume)
    u32         soundChannel  // Sound channel identifier
}
```

7.6b SNEM — Sound Emitters

Tag: `0x4D454E53` · **Variable-sized objects**

Note: Like SNDS, this chunk is not present in any known shipped model but readers exist in the internal game code. Sound emitters are node-based objects that can emit sounds at specific animation frames via KSEK tracks.

```
SNEM {
    SoundEmitter[?] soundEmitters // Parse using inclusiveSize
}

SoundEmitter {
    u32    inclusiveSize // Total size including this field
    Node   node           // Base node data with transform
    (KSEK) // Sound track: u32 values
}
```

7.7 MTLS — Materials

Tag: `0x534C544D` · **Variable-sized objects**

```
MTLS {
    Material[?] materials // Parse using inclusiveSize
}
```

Material

```
Material {
    u32    inclusiveSize // Total size including this field
```

```

    u32      priorityPlane
    u32      flags

    // version > 800 AND version < 1100 only:
    char[80]  shader          // Shader name (e.g. "Shader_HD_DefaultUnit")

    FourCC    "LAYS"          // Layer sub-chunk tag (0x5359414C)
    u32      layerCount
    Layer[layerCount] layers
}

```

Important version gate: The `shader` field exists only when `version > 800 && version < 1100`. In `version ≥ 1100`, the `shader` field was removed and the layer system was redesigned with the SubTexture system (see Layer below).

Layer

```

Layer {
    u32      inclusiveSize    // Total size including this field
    u32      filterMode       // See FilterMode enum
    u32      shadingFlags     // See ShadingFlags bitfield
    u32      textureId        // Index into TEXS
    u32      textureAnimationId // Index into TXAN, or 0xFFFFFFFF (-1)
    u32      coordId          // UV set index in geoset
    f32      alpha            // Layer opacity (0.0-1.0)

    // version > 800:
    f32      emissiveGain     // Emissive intensity; default 1.0 (not 0.0)

    // version > 900:
    float[3] fresnelColor     // RGB; default (1, 1, 1)
    f32      fresnelOpacity
    f32      fresnelTeamColor

    // version >= 1100 (SubTexture system):
    u32      shader          // ShaderType enum (0 = SD, 1 = HD, ...)
    u32      numSubTextures
    SubTexture[numSubTextures] subTextures

    // Animation tracks:
    (KMTF)                                // version < 1100 only
    (KMTA)
    (KMTE)                                // version > 800
    (KFC3)                                // version > 900
    (KFCA)                                // version > 900
    (KFTC)                                // version > 900
}

```

Version gate — fresnel fields: `emissiveGain` exists when `version > 800`, but `fresnelColor`, `fresnelOpacity`, and `fresnelTeamColor` exist only when `version > 900`. Gating all four on

version > 800 (a common mistake) makes a v900 layer consume 16 bytes of the following track chunk as fresnel data, yielding NaN fresnel values and a desynchronised track stream. The fresnel *track* chunks (KFC3/KFCA/KFTC) share the same **version > 900** gate.

shader field (version ≥ 1100): A **u32** holding a **ShaderType** enum value — **not** a boolean **isHD**. SD layers use **0**, HD (PBR) layers use **1**; other values exist for engine-internal post-process shaders (see **ShaderType** enum).

SubTexture (version ≥ 1100)

```
SubTexture {
    u32      textureId      // Index into TEXTS
    u32      slot           // Texture slot (see SubTextureSlot enum)
    (KMTF)           // Per-subtexture texture ID animation
}
```

When **version >= 1100**, the KMTF track is stored per-SubTexture rather than on the Layer itself.

SubTextureSlot Enum

HD layers in Reforged use a PBR (Physically Based Rendering) texture pipeline. Each slot identifies the role of the texture in the shading model. An HD layer (**shader == 1**) typically has 6 SubTextures — one per slot — while SD layers (**shader == 0**) have a single SubTexture at slot 0.

Value	Name	Description
0	Diffuse	Albedo / base color map. In SD layers this is the only texture.
1	Normal	Tangent-space normal map. Default: Textures/normal.blp (flat normal).
2	ORM	Packed Occlusion (R), Roughness (G), Metallic (B) map. Default: Textures/ORM.blp .
3	Emissive	Emissive / self-illumination map. Default: Textures/Black32.blp (no emission).
4	Team Color	Replaceable team color texture. Typically references a texture entry with an empty filename and replaceableId = 1 .
5	Environment Map	Cube/sphere environment reflection map. Always ReplaceableTextures/EnvironmentMap.blp in the corpus.

Corpus validation (5081 v1200 files, 86321 SubTextures):

- **14,076 layers** use the full 6-slot HD set **{0, 1, 2, 3, 4, 5}**.
- **1,349 layers** use a single slot 0 (SD diffuse-only).
- **82 layers** use **{0, 2, 3, 4, 5}** (HD without normal map).
- Slots 1–5 never appear without slot 0. Slot values outside 0–5 were not observed.

ShaderType Enum (Layer **shader**, version ≥ 1100)

The **u32 shader** field selects the shading pipeline for the layer. Models on disk use only **0** (SD) and **1** (HD); the remaining values name engine-internal shaders (post-processing, terrain, UI, ...) and are listed for completeness.

Value	Name	Notes
0	SD	Classic fixed-function shading; single diffuse SubTexture at slot 0.
1	HD	Reforged PBR shading; the 6-slot SubTexture set.
2	SDOnHD	SD content rendered through the HD pipeline.
3–25	<i>(engine-internal)</i>	Terrain, Water, Fog, Foliage, Sprite, post-process (DepthOfField, Bloom*, GaussianBlur, Tonemap, CMAA*), Distortion, Crystal, ImGui, ... — not used by on-disk model layers.

FilterMode Enum

Value	Name	Description
0	None	Opaque
1	Transparent	Alpha-tested (threshold ~0.75)
2	Blend	Standard alpha blending
3	Additive	Additive blending ($\text{src alpha} \times \text{src} + \text{dst}$)
4	AddAlpha	Add alpha (same as additive in practice)
5	Modulate	Multiply ($\text{zero} \times \text{dst} + \text{src color} \times \text{dst}$)
6	Modulate2x	Modulate 2×

ShadingFlags Bitfield

Bit	Hex	Description
0	0x1	Unshaded
1	0x2	Sphere environment map
2	0x4	Wrap width (unknown)
3	0x8	Wrap height (unknown)
4	0x10	Two sided
5	0x20	Unfogged
6	0x40	No depth test
7	0x80	No depth set
8	0x100	Unlit (version > 800)

7.8 TXAN — Texture Animations

Tag: `0x4E415854` · **Variable-sized objects**

```
TXAN {
    TextureAnimation[?] animations // Parse using inclusiveSize
}

TextureAnimation {
    u32          inclusiveSize
    (KTAT)                               // Translation: float[3]
    (KTAR)                               // Rotation: float[4] (quaternion XYZW)
    (KTAS)                               // Scaling: float[3]
}
```

Note: KTAR uses a **quaternion** `float[4]` rotation, matching GhostWolf's original specification.

7.9 GEOS — Geosets

Tag: `0x534F4547` · **Variable-sized objects**

```
GEOS {
    Geoset[?] geosets // Parse using inclusiveSize
}
```

Geoset

```
Geoset {
    u32          inclusiveSize

    // Vertex positions
    FourCC      "VRTX" // 0x58545256
    u32          vertexCount
    float[vertexCount * 3] positions

    // Vertex normals
    FourCC      "NRMS" // 0x534D524E
    u32          normalCount
    float[normalCount * 3] normals

    // Primitive type groups
    FourCC      "PTYP" // 0x50595450
    u32          faceTypeGroupCount
    u32[faceTypeGroupCount] faceTypeGroups // Usually 4 (triangles)

    // Primitive count groups
    FourCC      "PCNT" // 0x544E4350
    u32          faceGroupCount
```

```

    u32[faceGroupCount] faceGroups

    // Face indices
    FourCC      "PVTX"                // 0x58545650
    u32          faceCount
    u16[faceCount] faces

    // Vertex group assignments
    FourCC      "GNDX"                // 0x58444E47
    u32          vertexGroupCount
    u8[vertexGroupCount] vertexGroups

    // Matrix group sizes
    FourCC      "MTGC"                // 0x4347544D
    u32          matrixGroupCount
    u32[matrixGroupCount] matrixGroups

    // Matrix indices
    FourCC      "MATS"                // 0x5354414D
    u32          matrixIndexCount
    u32[matrixIndexCount] matrixIndices

    u32          materialId            // Index into MTLS
    u32          selectionGroup
    u32          selectionFlags

    // version > 800:
    u32          lod                   // Level of detail
    char[80]     lodName               // LOD name

    Extent       extent                // Geoset bounds
    u32          sequenceExtentCount
    Extent[sequenceExtentCount] sequenceExtents

    // version > 800 (optional sub-chunks):
    (Tangents)
    (Skin)

    // Texture coordinate sets
    FourCC      "UVAS"                // 0x53415655
    u32          uvSetCount
    TextureCoordinateSet[uvSetCount] uvSets
}

```

Tangents Sub-Chunk (version > 800)

```

Tangents {
    FourCC      "TANG"                // 0x474E4154
    u32          count
}

```

```
float[count * 4] tangents // x, y, z, handedness (w)
}
```

Skin Sub-Chunk (version > 800)

```
Skin {
  FourCC      "SKIN"           // 0x4E494B53
  u32         count           // Byte count
  u8[count]   data            // Groups of 8 bytes: 4 bone indices + 4
  weights
}
```

The SKIN data replaces the GNDX/MTGC/MATS skinning system. Each vertex uses 8 bytes: 4 **u8** bone indices followed by 4 **u8** weights (divide by 255.0 to normalize; weights should sum to 1.0).

TextureCoordinateSet

```
TextureCoordinateSet {
  FourCC      "UVBS"           // 0x53425655
  u32         count
  float[count * 2] coordinates // u, v pairs
}
```

Primitive Types (faceTypeGroups values)

Value	Type
0	Points
1	Lines
2	Line loop
3	Line strip
4	Triangles
5	Triangle strip
6	Triangle fan
7	Quads
8	Quad strip
9	Polygons

In practice, virtually all models use type 4 (triangles).

7.10 GEOA — Geoset Animations

Tag: 0x414F4547 · **Variable-sized objects**

```
GEOA {
    GeosetAnimation[?] animations // Parse using inclusiveSize
}

GeosetAnimation {
    u32      inclusiveSize
    f32      alpha              // Static alpha value
    u32      flags              // 0x1 = color animation
    float[3] color              // Static color (BGR order)
    u32      geosetId           // Index of affected geoset

    (KGAO)                                // Alpha animation: f32
    (KGAC)                                // Color animation: float[3]
}
```

7.11 BONE — Bones

Tag: 0x454E4F42 · **No inclusiveSize** — compute from Node

```
BONE {
    Bone[?] bones
}

Bone {
    Node      node
    u32      geosetId           // Associated geoset, or 0xFFFFFFFF
    u32      geosetAnimationId // Associated geoset animation, or 0xFFFFFFFF
}
```

Size computation: Each Bone is `node.inclusiveSize + 8` bytes.

7.12 LITE — Lights

Tag: 0x4554494C · **Variable-sized objects**

```
LITE {
    Light[?] lights // Parse using inclusiveSize
}

Light {
    u32      inclusiveSize
    Node      node
    u32      type      // See LightType enum
    f32      attenuationStart
```

```
f32      attenuationEnd
float[3]  color           // RGB
f32      intensity
float[3]  ambientColor    // RGB
f32      ambientIntensity

// version >= 1200:
f32      shadowIntensity

(KLAS)           // Attenuation start: f32
(KLAE)           // Attenuation end: f32
(KLAC)           // Color: float[3]
(KLAI)           // Intensity: f32
(KLBI)           // Ambient intensity: f32
(KLBC)           // Ambient color: float[3]
(KLAV)           // Visibility: f32
}
```

LightType Enum

Value	Type
0	Omni
1	Directional
2	Ambient

7.13 HELP — Helpers

Tag: 0x504C4548 · No inclusiveSize

```
HELP {
  Helper[?] helpers
}

Helper {
  Node      node
}
```

Size computation: Each Helper is exactly node.inclusiveSize bytes.

7.14 ATCH — Attachments

Tag: 0x48435441 · Variable-sized objects

```
ATCH {
  Attachment[?] attachments    // Parse using inclusiveSize
}
```

```
Attachment {
    u32      inclusiveSize
    Node     node
    char[260] path          // Attachment model path
    u32      attachmentId

    (KATV)          // Visibility: f32
}
```

7.15 PIVT — Pivot Points

Tag: 0x54564950 · **Element size:** 12 bytes · **Count:** size / 12

```
PIVT {
    float[size / 4] points          // Groups of 3 floats (x, y, z)
}
```

Each pivot point is 12 bytes ($3 \times \text{f32}$). There is one pivot point per node (bones, helpers, lights, attachments, emitters, etc.) plus one per camera.

7.16 PREM — Particle Emitters (Model)

Tag: 0x4D455250 · **Variable-sized objects**

Emitters that spawn model instances.

```
PREM {
    ParticleEmitter[?] emitters    // Parse using inclusiveSize
}

ParticleEmitter {
    u32      inclusiveSize
    Node     node
    f32      emissionRate
    f32      gravity
    f32      longitude
    f32      latitude
    char[260] spawnModelFileName  // Model to emit
    f32      lifespan
    f32      initialVelocity

    (KPEE)          // Emission rate: f32
    (KPEG)          // Gravity: f32
    (KPLN)          // Longitude: f32
    (KPLT)          // Latitude: f32
    (KPEL)          // Lifespan: f32
    (KPES)          // Speed: f32
}
```



```

    (KPEV)                                // Visibility: f32
}

```

7.17 PRE2 — Particle Emitters 2 (Quad)

Tag: `0x32455250` · **Variable-sized objects**

Emitters that spawn textured quads (billboards).

```

PRE2 {
    ParticleEmitter2[?] emitters    // Parse using inclusiveSize
}

ParticleEmitter2 {
    u32        inclusiveSize
    Node        node
    f32        speed
    f32        variation
    f32        latitude
    f32        gravity
    f32        lifespan
    f32        emissionRate
    f32        length
    f32        width

    u32        filterMode           // 0=blend, 1=additive, 2=modulate,
3=modulate2x, 4=alphaKey
    u32        rows                 // Texture atlas rows
    u32        columns              // Texture atlas columns
    u32        headOrTail           // 0=head, 1=tail, 2=both

    f32        tailLength
    f32        time                 // Mid-point time (0.0-1.0)

    float[3]    segmentColor[3]    // 3 segments × RGB (9 floats total)
    u8[3]       segmentAlpha        // 3 alpha values (0-255)
    float[3]    segmentScaling      // 3 scale factors

    u32[3]      headInterval        // Start/mid/end lifecycle intervals
    u32[3]      headDecayInterval
    u32[3]      tailInterval
    u32[3]      tailDecayInterval

    u32        textureId            // Index into TEXS
    u32        squirt               // 1 = burst mode
    u32        priorityPlane
    u32        replaceableId

    (KP2S)      // Speed: f32
    (KP2R)      // Variation: f32
    (KP2L)      // Latitude: f32
    (KP2G)      // Gravity: f32
}

```

```

    (KP2E)          // Emission rate: f32
    (KP2N)          // Length: f32
    (KP2W)          // Width: f32
    (KP2V)          // Visibility: f32
}

```

Note: `length` is written before `width`. Some older specifications had these swapped.

7.18 RIBB — Ribbon Emitters

Tag: `0x42424952` · **Variable-sized objects**

Emitters that produce connected line segments forming ribbon-like quads.

```

RIBB {
    RibbonEmitter[?] emitters    // Parse using inclusiveSize
}

RibbonEmitter {
    u32      inclusiveSize
    Node     node
    f32      heightAbove
    f32      heightBelow
    f32      alpha
    float[3] color              // RGB
    f32      lifespan
    u32      textureSlot
    u32      emissionRate
    u32      rows
    u32      columns
    u32      materialId        // Index into MTLS
    f32      gravity

    (KRHA)          // Height above: f32
    (KRHB)          // Height below: f32
    (KRAL)          // Alpha: f32
    (KRCO)          // Color: float[3]
    (KRTX)          // Texture slot: u32
    (KRVS)          // Visibility: f32
}

```

7.19 EVTS — Event Objects

Tag: `0x53545645` · **No inclusiveSize**

Event objects trigger sound effects, footprints, blood splats, and other environmental effects at specific animation frames.

```

EVTS {
    EventObject[?] objects
}

EventObject {
    Node        node
    KEVT        tracks           // Event timing data (see Section 8.3)
}

```

Size computation: Each EventObject is `node.inclusiveSize` + the KEVT block size. See [Section 8.3](#) for the KEVT format.

7.20 CAMS — Cameras

Tag: `0x534D4143` · **Variable-sized objects**

```

CAMS {
    Camera[?] cameras           // Parse using inclusiveSize
}

Camera {
    u32        inclusiveSize
    char[80]   name
    float[3]   position         // Camera position (x, y, z)
    f32        fieldOfView      // Radians
    f32        farClippingPlane
    f32        nearClippingPlane
    float[3]   targetPosition   // Look-at target (x, y, z)

    (KCTR)     // Position: float[3]
    (KCRL)     // Rotation: f32 (scalar angle)
    (KTTR)     // Target position: float[3]
}

```

7.21 CLID — Collision Shapes

Tag: `0x44494C43` · **No inclusiveSize**

```

CLID {
    CollisionShape[?] shapes
}

CollisionShape {
    Node        node
    u32         type           // See ShapeType enum

    float[vertexCount * 3] vertices // See vertex counts below
}

```

```

    // type == Sphere (2) or type == Cylinder (3):
    f32      radius
}

```

ShapeType Enum and Vertex Counts

Value	Type	Vertex Count	Extra
0	Box	2	Two corners (min, max)
1	Plane	2	Two corners
2	Sphere	1	Center point + radius
3	Cylinder	2	Two endpoints + radius

Size computation: Node `inclusiveSize` + 4 (type) + vertexCount × 12 (vertices) + optional 4 (radius if sphere or cylinder).

7.22 BPOS — Bind Poses

Tag: `0x534F5042` · **Version** > 800

```

BPOS {
    u32      count           // Number of bind pose matrices
    float[count * 12] matrices // 3x4 row-major matrices (48 bytes each)
}

```

Each matrix is a 3×4 affine transform (3 rows × 4 columns = 12 floats). There is one bind pose per node (including cameras), so the count typically equals the PIVT count + camera count.

7.23 FAFX — Face Effects

Tag: `0x58464146` · **Element size:** 340 bytes · **Count:** `size / 340` · **Version** > 800

```

FAFX {
    FaceEffect[size / 340] effects
}

FaceEffect {
    // 340 bytes
    char[80]    target // Target name (e.g., "Node")
    char[260]   path   // FaceFX file path (.facefx)
}

```

7.24 CORN — Popcorn Emitters

Tag: `0x4E524F43` · **Variable-sized objects** · **Version** > 800

Emitters that use the PopcornFX particle runtime.

```
CORN {
    CornEmitter[?] emitters          // Parse using inclusiveSize
}

CornEmitter {
    u32      inclusiveSize
    Node     node
    f32      lifeSpan
    f32      emissionRate
    f32      speed
    float[4] color                  // RGBA (4 floats)
    u32      replaceableId
    char[260] path                  // Effect file path (.pkfx)
    char[260] animVisibilityGuide // Sequence-based visibility rules (see below)

    (KPPA)          // Alpha: f32
    (KPPC)          // Color: float[4] (RGBA)
    (KPPE)          // Emission rate: f32
    (KPPL)          // Lifespan: f32
    (KPPS)          // Speed: f32
    (KPPV)          // Visibility: f32
}
```

Important: The KPPC color track uses `float[4]` (RGBA), not `float[3]` (RGB). This differs from GhostWolf's specification and from the KRCO ribbon emitter color track which uses `float[3]`.

animVisibilityGuide

The `animVisibilityGuide` field is a comma-separated list of `Key=Value` rules that control when the PopcornFX emitter is active, based on the current animation sequence being played by the model. This provides a declarative way to tie particle emission to WC3's animation state machine without requiring per-frame KPPV visibility keyframes in every sequence.

Format: `SequenceName=State[, SequenceName=State, ...]`

The field is a null-terminated ASCII string within a 260-byte buffer. It may be empty (all zeroes), meaning the emitter is always active with no sequence-based toggling.

Values (case-insensitive in practice):

Value	Meaning
on	Emitter is active during sequences matching this key
off	Emitter is suppressed during sequences matching this key
On / Off	Case variants observed in corpus (equivalent to lowercase)
FStart/On-FEnd/Off	Frame-range control: on at frame start, off at frame end (rare, 2 occurrences)

Key names correspond to WC3 animation sequence names as defined in the SEQS chunk. The special key **Always** acts as a default rule. Common patterns:

Pattern	Meaning	Example
Always=on, Death=off	On by default, off during death	Persistent auras, weapon glow
Always=off, Death=on	Off by default, on during death	Death explosions, blood splatter
Always=off, Birth=on	Off by default, on during birth	Building construction FX
Always=off, Stand Work=on	Off by default, on when building is working	Building work activity FX
Always=on, Death=off, Decay=off	On by default, off during death/decay	Common for spell effects
Always=On, Death=Off, Dissipate=Off, Portrait=Off	Hero glow: off during death/dissipate/portrait	Hero aura particles
Always=off, Stand=on	Off by default, on during idle	Brazier flames, ambient FX
Always=off, Walk=on	Off by default, on when walking	Dust trails

Corpus statistics (5,081 v1200 files):

- **1,412 files** contain CORN chunks with **2,747 total emitters**
- **2,625** emitters (95.6%) have non-empty guides; **122** (4.4%) are empty
- **149 unique** guide strings observed; **96 unique** key names
- **5 value states:** **on** (2,569×), **off** (2,824×), **On** (222×), **Off** (647×), **FStart/On-FEnd/Off** (2×)

Most frequent guide strings:

Count	Guide String
430	Always=off, Death=on
321	Always=on, Death=off, Decay=off
286	Always=on, Death=off
225	Always=off, Birth=on
195	Always=On, Death=Off, Dissipate=Off, Portrait=Off
128	Always=off, Stand Work=on
122	(empty)
90	Always=off, Stand=on
79	Always=on

Count Guide String

59 Always=off, Walk=on

Key frequency (top 20):

Key	Count	Description
Always	2,492	Default state (on/off baseline)
Death	1,491	Death animation sequence
Decay	380	Corpse decay sequence
Dissipate	333	Hero/summoned unit dissipate
Portrait	308	Portrait/UI view mode
Birth	264	Construction/birth sequence
Stand	150	Idle standing animation
Stand Work	148	Building working (training/researching)
Walk	64	Walk animation
Spell	45	Spell cast animation
Attack	28	Attack animation
Death Alternate	27	Alternate death animation
Stand Channel	26	Channeling spell idle
Death 1	22	Death variation 1
Morph	22	Morph/transform animation
Spell Slam	18	Slam spell animation
Spell 1	16	Spell variation 1
Morph Alternate	16	Alternate morph animation
Stand Work Gold	14	Gold-gathering work animation
Walk 1	14	Walk variation 1

Note: Some corpus entries contain typos (Alwyas, Allways, deacy for Decay) and inconsistent casing (always vs Always, Off vs off). The engine likely performs case-insensitive matching. Some entries have trailing `\r\n` sequences or extra whitespace, indicating imprecise editor tooling.

8. Animation Tracks

8.1 Track Structure

All animation tracks (except KEVT) follow a uniform structure. The track tag identifies both the parent object and the data type of keyframes.

```
TrackChunk {
    FourCC      tag           // Track identifier (e.g., KGTR, KMTA, etc.)
    u32         keyCount      // Number of keyframes
    u32         interpolationType // See InterpolationType enum
    u32         globalSequenceId // Index into GLBS, or 0xFFFFFFFF (-1) for
    none

    Keyframe[keyCount] keyframes
}
```

Keyframe

```
Keyframe {
    i32      frame           // Time in milliseconds (may be negative in
    some models)
    T        value           // Keyframe value (type depends on track tag)

    // interpolationType > 1 (Hermite or Bezier):
    T        inTan           // In-tangent
    T        outTan          // Out-tangent
}
```

InterpolationType Enum

Value	Type	Description
0	None	No interpolation (stepped)
1	Linear	Linear interpolation
2	Hermite	Hermite spline (uses inTan/outTan)
3	Bezier	Bezier spline (uses inTan/outTan)

8.2 Track Tag Reference

The following table lists all track tags, their data type **T**, parent object, and semantic meaning.

Node Tracks

Tag	Data Type	Description
KGTR	float[3]	Translation (x, y, z)
KGRT	float[4]	Rotation (quaternion: x, y, z, w)

Tag	Data Type	Description
KGSC	<code>float[3]</code>	Scaling (x, y, z)

Layer Tracks

Tag	Data Type	Description	Version
KMTF	<code>u32</code>	Texture ID	All (per-SubTexture in ≥ 1100)
KMTA	<code>f32</code>	Alpha	All
KMTE	<code>f32</code>	Emissive gain	> 800
KFC3	<code>float[3]</code>	Fresnel color (RGB)	> 900
KFCA	<code>f32</code>	Fresnel alpha	> 900
KFTC	<code>f32</code>	Fresnel team color	> 900

Texture Animation Tracks

Tag	Data Type	Description
KTAT	<code>float[3]</code>	Translation (u, v, w)
KTAR	<code>float[4]</code>	Rotation (quaternion XYZW)
KTAS	<code>float[3]</code>	Scaling (u, v, w)

Note: KTAR uses a quaternion `float[4]`, same as node rotation KGRT.

Geoset Animation Tracks

Tag	Data Type	Description
KGAO	<code>f32</code>	Alpha
KGAC	<code>float[3]</code>	Color (BGR)

Light Tracks

Tag	Data Type	Description
KLAS	<code>f32</code>	Attenuation start
KLAE	<code>f32</code>	Attenuation end
KLAC	<code>float[3]</code>	Color (RGB)
KLAI	<code>f32</code>	Intensity
KLBI	<code>f32</code>	Ambient intensity
KLBC	<code>float[3]</code>	Ambient color (RGB)

Tag	Data Type	Description
KLAV	f32	Visibility

Attachment Tracks

Tag	Data Type	Description
KATV	f32	Visibility

Particle Emitter Tracks (PREM)

Tag	Data Type	Description
KPEE	f32	Emission rate
KPEG	f32	Gravity
KPLN	f32	Longitude
KPLT	f32	Latitude
KPEL	f32	Lifespan
KPES	f32	Speed
KPEV	f32	Visibility

Particle Emitter 2 Tracks (PRE2)

Tag	Data Type	Description
KP2S	f32	Speed
KP2R	f32	Variation
KP2L	f32	Latitude
KP2G	f32	Gravity
KP2E	f32	Emission rate
KP2N	f32	Length
KP2W	f32	Width
KP2V	f32	Visibility

Ribbon Emitter Tracks

Tag	Data Type	Description
KRHA	f32	Height above
KRHB	f32	Height below

Tag	Data Type	Description
KRAL	f32	Alpha
KRCO	float[3]	Color (RGB)
KRTX	u32	Texture slot
KRVS	f32	Visibility

Camera Tracks

Tag	Data Type	Description
KCTR	float[3]	Position (x, y, z)
KCRL	f32	Target rotation (scalar angle)
KTTR	float[3]	Target position (x, y, z)

Popcorn Emitter Tracks (CORN)

Tag	Data Type	Description
KPPA	f32	Alpha
KPPC	float[4]	Color (RGBA)
KPPE	f32	Emission rate
KPPL	f32	Lifespan
KPPS	f32	Speed
KPPV	f32	Visibility

Sound Emitter Tracks (SNEM)

Tag	Data Type	Description
KSEK	u32	Sound event track

Note: KSEK follows the standard track structure (with interpolation type and global sequence ID in the header), unlike KEVT which has a unique layout.

8.3 KEVT — Event Tracks

Event object tracks have a unique structure that differs from all other track types. They store only frame times with no interpolation or value data — there is no `interpolationType` field.

```
KEVT {
  FourCC      "KEVT"           // 0x5456454B
  u32         trackCount       // Number of event times
```

```
    u32      globalSequenceId    // Index into GLBS, or 0xFFFFFFFF (-1)
    i32[trackCount] tracks      // Frame times (ms) when the event fires
}
```

Field order: `globalSequenceId` sits **between** `trackCount` and the track data — the same position it occupies in a standard track header, just with the `interpolationType` field omitted. (Verified against war3.w3mod game assets and the mdx-m3-viewer reference reader.) The track times are signed `i32`, consistent with keyframe `frame` values elsewhere.

9. Node Structure

The `Node` is a shared structure embedded in Bones, Lights, Helpers, Attachments, Particle Emitters, Ribbon Emitters, Event Objects, Collision Shapes, and Popcorn Emitters. It defines the object's place in the scene hierarchy and its animation channels.

```
Node {
    u32      inclusiveSize      // Total size of this Node including this
    field
        char[80]  name          // Node name
        u32      objectId       // Unique node ID
        u32      parentId       // Parent node ID, or 0xFFFFFFFF (-1) for root
        u32      flags          // See NodeFlags bitfield

        (KGTR)                                // Translation: float[3]
        (KGRT)                                // Rotation: float[4] (quaternion)
        (KGSC)                                // Scaling: float[3]
}
```

NodeFlags Bitfield

The `flags` field encodes both the node type **and** behavioral flags.

Node Type Flags

Bit	Hex	Type
8	0x100	Bone
9	0x200	Light
10	0x400	Event object
11	0x800	Attachment
12	0x1000	Particle emitter
13	0x2000	Collision shape
14	0x4000	Ribbon emitter

Bit	Hex	Type
17	0x20000	Line emitter / Particle emitter 2

If none of the type bits are set (`flags & 0xFF00 == 0`), the node is a Helper.

Behavioral Flags

Bit	Hex	Description
0	0x1	Don't inherit translation
1	0x2	Don't inherit rotation
2	0x4	Don't inherit scaling
3	0x8	Billboarded
4	0x10	Billboarded lock X
5	0x20	Billboarded lock Y
6	0x40	Billboarded lock Z
7	0x80	Camera anchored
15	0x8000	Emitter uses MDL (PREM) / Unshaded (PRE2)
16	0x10000	Emitter uses TGA (PREM) / Sort primitives far Z (PRE2)
18	0x40000	Unfogged
19	0x80000	Model space
20	0x100000	XY quad

Note on multiple roots: MDX models commonly have multiple root nodes (`parentId = 0xFFFFFFFF`). For example, unit models often have separate bone hierarchies for the living model and its corpse/decay animation.

10. Version Differences

This section summarizes the structural changes introduced in each version.

Version 800 (Classic)

The base format. All structures documented above without version conditions apply.

Version 900 (Reforged Beta)

Added to existing structures:

- **Material:** `char[80]` `shader` field after `flags`
- **Layer:** `emissiveGain` field after `alpha`; KMTE track

- **Geoset:** `lod (u32)` and `lodName (char[80])` after `selectionFlags`; optional TANG and SKIN sub-chunks

New chunks:

- **BPOS:** Bind pose matrices
- **FAFX:** Face animation effects (FaceFX)
- **CORN:** Popcorn particle emitters

Version 1000 (Reforged — First Release)

Added to existing structures:

- **Layer:** `fresnelColor[3]`, `fresnelOpacity`, `fresnelTeamColor` fields after `emissiveGain`; KFC3, KFCA, KFTC Fresnel animation tracks

The fresnel **fields** and fresnel **tracks** were added together in v1000. A v900 layer has `emissiveGain` but no fresnel data at all.

Version 1100 (Reforged — 2.0.0)

Major layer system change:

- **Material:** `shader` string field **removed** (was only for $800 < \text{version} < 1100$)
- **Layer:** SubTexture system added — `shader (ShaderType u32)`, `numSubTextures`, and per-subtexture `{textureId, slot, KMTF}`
- **Layer:** KMTF track moved from Layer level to per-SubTexture

Version 1200 (Reforged Current)

Added to existing structures:

- **Light:** `shadowIntensity` field (`f32`) after `ambientIntensity`

11. Implementation Notes

Byte Order

All multi-byte values are **little-endian**.

String Fields

Fixed-length string fields (`char[80]`, `char[260]`) are null-padded. The actual string content ends at the first null byte; remaining bytes should be zero. When writing, pad with null bytes to the full field length.

Animation Timing

Warcraft III animations use millisecond timing. At 60 FPS, advance animation counters by $1000/60 \approx 16.67$ ms per frame. Keyframe times in tracks and sequence intervals are in milliseconds.

Rotations

- **Node rotations (KGRT):** Quaternions as `float[4]` in (x, y, z, w) order.
- **Texture animation rotations (KTAR):** Quaternions as `float[4]` in (x, y, z, w) order.
- **Camera rotations (KCRL):** Scalar `f32` angle.

Inclusive Size Fields

The `inclusiveSize` at the start of variable-sized objects includes the 4 bytes of the `inclusiveSize` field itself. For example, if `inclusiveSize = 100`, the object occupies exactly 100 bytes from the start of the `inclusiveSize` field (96 bytes of payload after it).

Geoset Sub-Chunk Tag-Size Pairs

Within a Geoset, sub-chunks like VRTX, NRMS, PTYP, etc. use a tag + count pattern (not tag + byte-size). The count represents the number of elements, not bytes. For example:

- VRTX count = number of vertices (each vertex is 3 floats = 12 bytes)
- PVTX count = number of face indices (each is 1 `u16` = 2 bytes)
- GNDX count = number of vertex groups (each is 1 `u8` = 1 byte)

Global Sequence IDs

A `globalSequenceId` of `0xFFFFFFFF` (-1 as signed) means the track uses normal sequence-based animation rather than a global sequence loop.

Parsing Tracks Within Objects

To determine whether optional animation tracks exist within an object, either:

1. **Size tracking:** Track bytes consumed vs. `inclusiveSize` — if bytes remain, the next 4 bytes may be a track tag.
2. **Tag peeking:** Read the next 4 bytes and check against known track tags for that object type. If it matches, parse it; otherwise, stop.

Approach 1 is more robust as it handles unknown track types gracefully.

Chunk Tag Constants

For reference, here are the FourCC tag values as `u32` little-endian:

MDLX = 0x584C444D	VERS = 0x53524556	MODL = 0x4C444F4D	
SEQS = 0x53514553	GLBS = 0x53424C47	TEXS = 0x53584554	
SNDS = 0x53444E53	SNEM = 0x4D454E53	MTLS = 0x534C544D	TXAN =
0x4E415854			
GEOS = 0x534F4547	GEOA = 0x414F4547	BONE = 0x454E4F42	
LITE = 0x4554494C	HELP = 0x504C4548	ATCH = 0x48435441	
PIVT = 0x54564950	PREM = 0x4D455250	PRE2 = 0x32455250	
RIBB = 0x42424952	EVTS = 0x53545645	CAMS = 0x534D4143	
CLID = 0x44494C43	BPOS = 0x534F5042	FAFX = 0x58464146	
CORN = 0x4E524F43			
LAYS = 0x5359414C	VRTX = 0x58545256	NRMS = 0x534D524E	

PTYP = 0x50595450	PCNT = 0x544E4350	PVTX = 0x58545650
GNDX = 0x58444E47	MTGC = 0x4347544D	MATS = 0x5354414D
TANG = 0x474E4154	SKIN = 0x4E494B53	UVAS = 0x53415655
UVBS = 0x53425655	KEVT = 0x5456454B	KSEK = 0x4B45534B

Track Tag Constants

KGTR = 0x5254474B	KGRT = 0x5452474B	KGSC = 0x4353474B
KMTF = 0x46544D4B	KMTA = 0x41544D4B	KMTE = 0x45544D4B
KFC3 = 0x3343464B	KFCA = 0x4143464B	KFTC = 0x4354464B
KTAT = 0x5441544B	KTAR = 0x5241544B	KTAS = 0x5341544B
KGAO = 0x4F41474B	KGAC = 0x4341474B	
KLAS = 0x53414C4B	KLAE = 0x45414C4B	KLAC = 0x43414C4B
KLAI = 0x49414C4B	KLBI = 0x49424C4B	KLBC = 0x43424C4B
KLAV = 0x56414C4B		
KATV = 0x5654414B		
KPEE = 0x4545504B	KPEG = 0x4745504B	KPLN = 0x4E4C504B
KPLT = 0x544C504B	KPEL = 0x4C45504B	KPES = 0x5345504B
KPEV = 0x5645504B		
KP2S = 0x5332504B	KP2R = 0x5232504B	KP2L = 0x4C32504B
KP2G = 0x4732504B	KP2E = 0x4532504B	KP2N = 0x4E32504B
KP2W = 0x5732504B	KP2V = 0x5632504B	
KRHA = 0x4148524B	KRHB = 0x4248524B	KRAL = 0x4C41524B
KRCO = 0x4F43524B	KRTX = 0x5854524B	KRVS = 0x5356524B
KCTR = 0x5254434B	KCRL = 0x4C52434B	KTTR = 0x5254544B
KPPA = 0x4150504B	KPPC = 0x4350504B	KPPE = 0x4550504B
KPPL = 0x4C50504B	KPPS = 0x5350504B	KPPV = 0x5650504B
KSEK = 0x4B45534B		

Appendix A — PopcornFX Integration

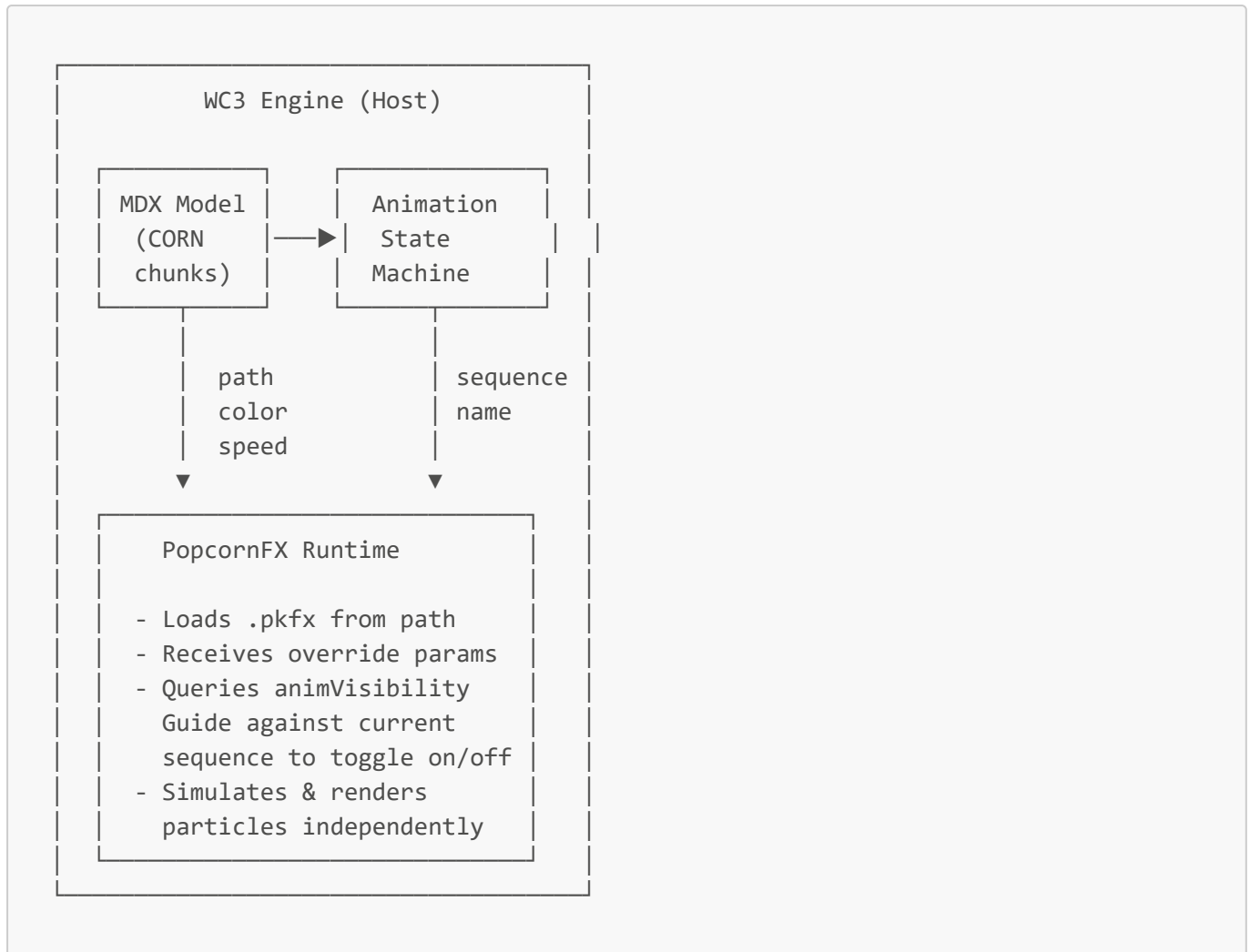
Overview

Warcraft III: Reforged (version > 800) replaced the classic particle system with **PopcornFX**, a commercial real-time particle effects middleware by Persistent Studios. Instead of the engine computing particle physics from inline parameters (as PREM and PRE2 do), CORN emitters delegate particle simulation entirely to the PopcornFX runtime, which loads external effect definition files.

This is a fundamental architectural shift: classic emitters (PREM, PRE2) store all particle behavior inside the MDX binary — spawn rates, physics, textures, color curves — while CORN emitters are lightweight bridge objects that point to an external `.pkfx` effect file and pass a small set of override parameters.

Runtime Architecture

The integration follows a host-plugin model:



1. The engine loads the MDX model and encounters one or more **CornEmitter** objects in the CORN chunk.
2. Each emitter's **Node** places it in the scene hierarchy (parented to bones, etc.), giving it a world-space transform.
3. The engine passes the emitter's **path** to the PopcornFX runtime, which loads the **.pkfx** effect definition.
4. Per-frame, the engine feeds the PopcornFX runtime the emitter's current transform (from the Node hierarchy), and the override parameters: **color**, **speed**, **emissionRate**, **lifeSpan** — including any animated values from KPPC/KPPS/KPPE/KPPL tracks.
5. The engine evaluates the **animVisibilityGuide** string against the currently playing animation sequence name. If the guide maps the current sequence to **off**, the emitter is suppressed; if **on**, it is active. The KPPV visibility track provides additional per-keyframe control.
6. The PopcornFX runtime handles all particle spawning, simulation, and rendering autonomously.

.pkfx Files

PopcornFX effect files (**.pkfx**) are **baked binary effect packages** produced by the PopcornFX Editor. They are not part of the MDX format — they are external assets loaded at runtime by the PopcornFX plugin.

A **.pkfx** file encapsulates:

Component	Description
Particle layers	One or more particle layers, each defining spawn shape, velocity, physics, and lifetime
Renderers	Billboard, ribbon, mesh, or light renderers with material/blend settings
Textures & atlases	Texture references (resolved relative to the game's asset root), sprite sheet definitions
Evolvers	Gravity, turbulence, collisions, attractors, and other physics modules
Samplers	Curve samplers for color-over-life, size-over-life, etc.
Events & triggers	Spawn-on-death, sub-effect triggers, sound hooks
Spatial layers	LOD settings, bounding volumes for culling

The MDX `CornEmitter` does **not** replicate any of this data — it only stores the path to the `.pkfx` file and a small set of host-side overrides.

Note: The `.pkfx` format is proprietary to PopcornFX and is not documented here. The PopcornFX SDK documentation covers the baked effect format internals.

Path Conventions

The `path` field in `CornEmitter` is a 260-byte null-terminated string containing a relative path to the `.pkfx` asset. Paths use forward slashes (/) as separators, though some corpus entries use backslashes (\). The path is relative to the game's asset root directory.

Directory structure observed in the corpus (2,747 emitters, 766 unique paths):

Top-Level Directory	Count	Description
<code>SharedFX/</code>	1,597	Reusable effects shared across many models
<code>Abilities/</code>	335	Spell and ability effects
<code>Units/</code>	312	Unit-specific effects (auras, weapon glow, etc.)
<code>Buildings/</code>	274	Building construction, work, and destruction FX
<code>Doodads/</code>	89	Doodad/prop effects and cinematic FX
<code>Objects/</code>	69	Item and object effects
<code>Environment/</code>	24	Environmental/weather effects
<code>UI/</code>	1	UI overlay effects

Typical path structure: `{Category}/{Subcategory}/{EffectName}/{EffectName}.pkfx`

Examples:

```

SharedFX/HUBuildingDeath_Small/HUBuildingDeath_Small.pkfx    (266 emitters)
SharedFX/Hero_Glow/Hero_Glow.pkfx                            (196 emitters)
SharedFX/HumanBuildingBirth_Small/HumanBuildingBirth_Small.pkfx (185 emitters)
Units/Demon/Infernal/Infernal.pkfx                           (20 emitters)
Abilities/Spells/Undead/FrostNova/FrostNovaTarget.pkfx       (spell FX)
Buildings/Human/AltarOfKings/AOK_Glow.pkfx                   (building FX)

```

The **SharedFX/** directory is heavily reused — a single effect like **HUBuildingDeath_Small.pkfx** is referenced by 266 different emitters across many building models, enabling a unified destruction look.

Corpus note: 26 paths (< 1%) lack the **.pkfx** extension (e.g., **SharedFX/Ship_Wake_Small/**), likely authoring errors that the engine resolves by appending a default extension or searching the directory. 7 paths contain absolute developer paths (**D:\Warcraft3\warcraft3\branches\v1.32.0.5\...**), leaked from development builds.

MDX-to-PopcornFX Data Flow

The **CornEmitter** fields map to the PopcornFX runtime as follows:

MDX Field	PopcornFX Role	Notes
path	Effect file to load	Resolved against asset root
color (float[4] RGBA)	Tint override	Passed to effect as an attribute sampler; many effects use black (0,0,0,1) as neutral
speed	Speed multiplier	Scales particle velocity in the effect
emissionRate	Emission rate multiplier	Scales spawn count
lifeSpan	Lifetime multiplier	Scales particle lifetime
replaceableId	Texture replacement ID	For team-color or replaceable texture effects (0 = none)
Node transform	World-space transform	The emitter inherits position/rotation/scale from the bone hierarchy
KPPA track	Animated alpha	Per-keyframe alpha override
KPPC track	Animated color	Per-keyframe RGBA color override
KPPE track	Animated emission rate	Per-keyframe emission rate override
KPPL track	Animated lifespan	Per-keyframe lifespan override
KPPS track	Animated speed	Per-keyframe speed override
KPPV track	Animated visibility	Per-keyframe visibility (0.0 = hidden, 1.0 = visible)

The static fields provide default values. When animation tracks are present, the tracked values override the static defaults at each keyframe.

Visibility Control

CORN emitters have a **two-layer visibility system** that is unique among MDX object types:

Layer 1 — `animVisibilityGuide` (sequence-level): A declarative string that maps animation sequence names to on/off states. The engine evaluates this against the model's current sequence (from SEQS) to determine whether the emitter should be active at all. See [Section 7.24](#) for the full guide format, key reference, and corpus statistics.

Layer 2 — KPPV track (keyframe-level): A standard `f32` animation track that provides per-keyframe visibility control within a sequence. Values of `0.0` suppress the emitter; `1.0` makes it visible.

The two layers combine: the `animVisibilityGuide` acts as a coarse gate (is this emitter relevant for the current animation?), while KPPV provides fine-grained per-frame control within active sequences.

This dual system lets artists achieve common patterns efficiently:

- **"Always on except during death"** — Set `Always=on`, `Death=off` in the guide instead of keyframing KPPV across every sequence.
- **"Only during spell cast, with fade-in"** — Set `Always=off`, `Spell=on` in the guide, then use a KPPV hermite track for the fade-in curve within the Spell sequence.

Corpus Statistics

Metric	Value
Files with CORN chunks	1,412 / 5,081 (27.8%)
Total CORN emitters	2,747
Unique .pkfx paths	766
Emitters with non-empty guide	2,625 (95.6%)
Emitters with empty guide	122 (4.4%)
Unique guide strings	149
Unique guide key names	96
Most-referenced effect	<code>SharedFX/HUBuildingDeath_Small/HUBuildingDeath_Small.pkfx</code> (266×)

Credits & References

- **WhiteoutLib** — C++ parser and writer implementation used as the primary reference for this specification
- **GhostWolf's MDX Specifications** — [HiveWorkshop thread](#) (2013–2019), the original community specification

- **Barncastle's MDXReForged** — C# implementation documenting Reforged-era changes (2019)
- **BlinkBoy** — Contributions on collision shape types, KEVT format, SND chunk, and various corrections
- **Magos' MDX Specifications** — Original reverse-engineering work that started community documentation
- **PopcornFX** — persistant-studios.com — Commercial particle effects middleware by Persistant Studios, integrated into WC3 Reforged

Key Corrections vs. Prior Specifications

Item	Old Spec	Corrected
KTAR data type	<code>f32</code> (scalar angle)	<code>float[4]</code> (quaternion XYZW)
KPPC data type	<code>float[3]</code> (RGB)	<code>float[4]</code> (RGBA)
KEVT field order	(v2.0 of this doc placed <code>globalSequenceId</code> after the track times)	<code>globalSequenceId</code> sits between <code>trackCount</code> and the track times — verified against game assets and mdx-m3-viewer
Material shader	Present when <code>version > 800</code>	Present when <code>version > 800 && version < 1100</code>
Light shadowIntensity	Not documented	Present when <code>version >= 1200</code>
Layer SubTexture system	Not documented	New in version ≥ 1100
Layer fresnel fields	Present when <code>version > 800</code> (with <code>emissiveGain</code>)	<code>emissiveGain</code> is <code>version > 800</code> ; <code>fresnelColor/fresnelOpacity/fresnelTeamColor</code> are <code>version > 900</code>
Layer $v \geq 1100$ field	<code>u32 isHD</code> (boolean)	<code>u32 shader</code> — a <code>ShaderType</code> enum (0 = SD, 1 = HD, ...)
Keyframe <code>frame</code>	Unsigned time	Signed <code>i32</code> ; negative frames occur in shipped models